Lab 06 — Symmetric vs. Asymmetric Encryption and Key Exchange

> **SY0-701 Objective 1.4** — Explain the importance of using appropriate cryptographic solutions.
>
> **Domain 1** — General Security Concepts (12%) · **Time:** ~25 min · **Course:** C628 CompTIA Security+ Exam Prep

Goal: Encrypt with a shared AES key, see why block mode matters, compare key lengths and speed, agree a secret with X25519 key exchange, and use GPG public-key encryption to reveal the hybrid design that TLS, S/MIME and PGP all rely on.

You'll build: An AES-256 encrypted file, an ECB-versus-CBC pattern comparison, an RSA-versus-ECC key size and speed table, a shared secret derived independently by Alice and Bob, and a GPG-encrypted message whose packet dump shows a public-key-wrapped AES session key.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 05 (hashing and signatures) is helpful; tools installed in step 1.

Background

- **Symmetric** encryption (AES, ChaCha20) uses one shared key to encrypt and decrypt. It is fast and suits bulk data, but the key must reach the other party safely.
- **Asymmetric** encryption (RSA, ECC) uses a key pair: encrypt with the recipient's **public key**, decrypt with their **private key**. It solves key distribution but is slow.
- **Key exchange** such as Diffie-Hellman (and its elliptic-curve forms ECDH and X25519) lets two parties derive the same secret without ever sending it. Ephemeral keys give **perfect forward secrecy**.
- **Key length** matters only *within* an algorithm: AES-256 beats AES-128, RSA-3072 beats RSA-2048, and a 256-bit ECC key is roughly as strong as 3072-bit RSA.
- Real systems are **hybrid**: asymmetric crypto protects a random symmetric session key, and that session key encrypts the data.

Steps

1. Install the tools

GnuPG provides public-key encryption; OpenSSL provides AES, RSA, ECC and key exchange.

```
apt-get update -qq && apt-get install -y gnupg openssl
mkdir -p /root/lab06 && cd /root/lab06
echo "top secret message: merger announced on Monday" > msg.txt
```

2. Encrypt and decrypt with a shared AES-256 key

One passphrase-derived key (PBKDF2 with a salt) both encrypts and decrypts. Fast and strong, but how do you get the passphrase to the recipient safely?

```
cd /root/lab06
openssl enc -aes-256-cbc -pbkdf2 -iter 200000 -salt -in msg.txt -out msg.aes -pass
pass:Lab1234!
od -c msg.aes | head -3
openssl enc -d -aes-256-cbc -pbkdf2 -iter 200000 -in msg.aes -pass pass:Lab1234!
```

The ciphertext starts with `Salted__`, followed by the random salt, then unreadable bytes.

3. Compare ECB and CBC block modes

ECB encrypts each 16-byte block independently, so identical plaintext blocks produce identical ciphertext and leak patterns. CBC chains blocks with an IV, so every block looks different.

```
cd /root/lab06
printf 'SECRET-PAYROLL!!%.0s' 1 2 3 4 > blocks.txt
KEY=$(openssl rand -hex 16); IV=$(openssl rand -hex 16)
openssl enc -aes-128-ecb -nosalt -K "$KEY" -in blocks.txt -out blocks.ecb
openssl enc -aes-128-cbc -nosalt -K "$KEY" -iv "$IV" -in blocks.txt -out blocks.cbc
echo "== ECB: repeated blocks =="; od -An -tx1 -w16 -v blocks.ecb | sort | uniq -c
echo "== CBC: all unique =="; od -An -tx1 -w16 -v blocks.cbc | sort | uniq -c
```

ECB shows one ciphertext block with a count of `4`; CBC shows five different blocks with a count of `1` each.

4. Compare key lengths and speed

Generate an RSA-3072 and an ECC P-256 key (similar strength) and compare public key sizes, then measure how many operations per second each can do compared with AES.

```
cd /root/lab06
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out rsa3072.key 2>/dev/null
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out ec256.key
echo "RSA-3072 public key bytes: $(openssl pkey -in rsa3072.key -pubout -outform DER | wc
-c)"
echo "ECC P-256 public key bytes: $(openssl pkey -in ec256.key -pubout -outform DER | wc -
c)"
openssl speed -seconds 1 rsa3072 ecdsap256 2>/dev/null | tail -4
openssl speed -seconds 1 -bytes 16384 -evp aes-256-gcm 2>/dev/null | tail -2
```

ECC achieves comparable security with a public key about five times smaller, and AES processes hundreds of megabytes per second while RSA manages only hundreds to thousands of operations.

5. Agree a shared secret with X25519 key exchange

Alice and Bob each create a key pair and swap only their public keys. Each combines their own private key with the other's public key and arrives at the same 256-bit secret, which never crossed the network.

```
cd /root/lab06
for p in alice bob; do openssl genpkey -algorithm X25519 -out "$p-x.key"; openssl pkey -in
"$p-x.key" -pubout -out "$p-x.pub"; done
openssl pkeyutl -derive -inkey alice-x.key -peerkey bob-x.pub -out alice-shared.bin
```

```
openssl pkeyutl -derive -inkey bob-x.key -peerkey alice-x.pub -out bob-shared.bin
sha256sum alice-shared.bin bob-shared.bin
echo "shared secret length: $(stat -c %s alice-shared.bin) bytes = $(( $(stat -c %s alice-shared.bin) * 8 )) bits"
```

Both lines show the same digest. That 256-bit value can now key AES-256: this is how TLS 1.3 sets up every session with ephemeral ECDHE.

6. Generate an asymmetric GPG key pair

Create a key pair for Alice: an RSA-3072 primary key for signing plus an RSA-3072 subkey for encryption (`[E]`). The public key can be published anywhere; the private key never leaves her keyring. (An empty passphrase is used only to keep the lab non-interactive.)

```
cd /root/lab06
gpg --list-keys alice@lab.local >/dev/null 2>&1 || gpg --batch --pinentry-mode loopback --passphrase '' --quick-generate-key 'Alice <alice@lab.local>' rsa3072 sign 1y
FPR=$(gpg --list-keys --with-colons alice@lab.local | awk -F: '/^fpr/{print $10; exit}')
gpg --list-keys alice@lab.local | grep -q '\[E\]' || gpg --batch --pinentry-mode loopback --passphrase '' --quick-add-key "$FPR" rsa3072 encr 1y
gpg --list-keys alice@lab.local
gpg --armor --export alice@lab.local > alice.pub
head -2 alice.pub
```

7. Encrypt to the public key and decrypt with the private key

Anyone holding `alice.pub` can encrypt to Alice; only Alice's private key can decrypt. This removes the key-distribution problem from step 2.

```
cd /root/lab06
gpg --batch --yes --trust-model always -r alice@lab.local -e -o msg.gpg msg.txt
od -c msg.gpg | head -2
gpg --batch --pinentry-mode loopback --passphrase '' -d msg.gpg 2>/dev/null
```

8. Inspect the hybrid structure of the GPG message

The packet dump shows the design: a public-key-encrypted packet carrying a random session key, then a symmetric (AES) encrypted data packet carrying the message.

```
cd /root/lab06
gpg --batch --pinentry-mode loopback --passphrase '' --list-packets msg.gpg 2>&1 | grep -E "pubkey enc packet|encrypted data packet|aead encrypted packet|session key|cipher|algo" | head -6
```

```
gpg: encrypted with rsa3072 key, ID ..., created ...
:pubkey enc packet: version 3, algo 1, keyid ...
:encrypted data packet:   (GnuPG 2.2)   or   :aead encrypted packet: cipher=9 aead=2 (GnuPG 2.4)
```

`algo 1` is RSA, and cipher `9` is AES-256. Small, slow RSA protects the 32-byte session key; fast AES protects the data.

Test it

```
cd /root/lab06
openssl enc -d -aes-256-cbc -pbkdf2 -iter 200000 -in msg.aes -pass pass:Lab1234! | grep -c
"merger"
od -An -tx1 -w16 -v blocks.ecb | sort | uniq -c | sort -rn | head -1 | awk '{print $1}'
[ "$(sha256sum < alice-shared.bin)" = "$(sha256sum < bob-shared.bin)" ] && echo "KEY
EXCHANGE OK"
gpg --batch --pinentry-mode loopback --passphrase '' -d msg.gpg 2>/dev/null | grep -c
"merger"
```

Expected: `1` (AES round trip works), `4` (ECB repeated one ciphertext block four times), `KEY EXCHANGE OK`, and `1` (GPG round trip works).

Troubleshooting

Symptom	Fix
`bad decrypt` from `openssl enc -d`	The passphrase, cipher and `-pbkdf2 -iter` value must match the encryption command exactly.
`gpg: decryption failed: No secret key`	The key was created in a different `GNUPGHOME` or user. Run `gpg --list-secret-keys` as root and regenerate the key in step 6 if it is missing.
`openssl speed` takes a long time or prints many lines	Keep `-seconds 1`; the command only benchmarks the algorithms named. Use `tail` to show the summary table.

Challenge

Show why **nonce reuse** is fatal for stream modes: encrypt two different 16-character messages with `-aes-128-ctr` using the *same* key and IV, XOR the two ciphertexts (for example with a short Python one-liner), and compare the result with the XOR of the two plaintexts. Explain why TLS 1.3 only allows AEAD ciphers such as AES-GCM and ChaCha20-Poly1305.

Reflection

Why does every real-world protocol in this lab (TLS, PGP, S/MIME) use asymmetric cryptography for the key and symmetric cryptography for the data, instead of one or the other alone?

Exam tips

- Symmetric = one shared key, fast, bulk data (AES, ChaCha20). Asymmetric = key pair, slow, key exchange and signatures (RSA, ECC).
- **Encrypt with the recipient's public key**; decrypt with the recipient's private key.
- Diffie-Hellman / ECDHE is **key exchange**, not encryption. Ephemeral keys give **perfect forward secrecy**.
- ECB leaks patterns and should never be chosen; GCM gives confidentiality and integrity together.

References

- NIST SP 800-57 Part 1 Recommendation for Key Management (key lengths) — https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final
- NIST FIPS 197 Advanced Encryption Standard — https://csrc.nist.gov/pubs/fips/197/final
- RFC 7748 Elliptic Curves for Security (X25519) — https://www.rfc-editor.org/rfc/rfc7748
- GnuPG manual — https://www.gnupg.org/documentation/manuals/gnupg/
